- `scope`—The *scopes* granted in the token. Scopes define what the user/app consented to (and is allowed to do). Taking the example from section 25.1, dogphotos.com may have requested the `photos.read` and `photos.edit` scopes, but if the user consented only to the `photos.read` scope, the `photos.edit` scope would not be in the JWT it receives for use with the Facebook photos API. It's up to the API itself to interpret what each scope means for the business logic of the request.
- `exp`—The expiration time of the token, after which it is no longer valid, expressed as the number of seconds since midnight on January 1, 1970 (known as the Unix timestamp).

An important point to realize is that JWTs are not encrypted. That means *anyone* can read the contents of a JWT by default. Another standard, JSON Web Encryption (JWE), can be used to wrap a JWT in an encrypted envelope that can't be read unless you have the key. Many identity providers include support for using JWEs with nested JWTs, and ASP.NET Core includes support for both out of the box, so it's something to consider.

Bearer tokens, access tokens, reference tokens, oh my!

The concept of a bearer token described in this section is a generic idea that can be used in several ways and for different purposes. You've already read about access tokens and identity tokens used in OpenID Connect. These are both bearer tokens; their different names describe the purpose of the token.

The following list describes some of the types of tokens you might read about or run into: